Replicating BMC: Safe In-kernel Caching and Pre-stack Processing for Memcached

Oleg Nedina
oleg.nedina@mail.polimi.it
Politecnico di Milano
Milan, Italy
https://github.com/Oleg-Nedina/BMC_Check

ABSTRACT

This report documents the project carried out for the Network Computing course at Politecnico di Milano. The goal of the project was to analyze, understand, and implement a reference research paper, aiming to replicate its results in a critical and intelligent manner. Consequently, this work is not intended to be a simple reproduction of the selected paper, but a comprehensive critical analysis of the presented tools, aimed at understanding their actual utility and limitations. In this report, I present the reference paper and the rationale behind its choice, the methodology used to analyze it, the initial findings obtained from this evaluation, and the subsequent experimental campaign, detailing all the caveats and technical hurdles encountered during replication.

1 INTRODUCTION

1.1 Selected Paper and Motivation

The selected paper is BMC [6], which introduces an in-kernel caching system for Memcached [4] built using eBPF and XDP [7].

The core idea is simple: using a caching system that completely bypasses the network stack of standard Linux servers, which becomes the primary performance bottleneck under the intensive workloads typically found in datacenters and warehouse-scale computers. The paper reviews alternative solutions like DPDK [12] or RDMA, explaining that although these traditional kernel-bypass technologies enable line-rate processing, they come with a high implementation cost. They force developers to entirely rewrite the application and prevent the use of standard Linux debugging and observability tools.

Here, BMC [6] comes into play as a solution positioned in this trade-off between maximum performance and operational costs. The authors propose a programmable in-kernel cache implemented entirely in eBPF and XDP. UDP GET requests are intercepted at the NIC driver's receive path—before the kernel allocates any network buffers (`sk_buff`)—and served directly from a BPF map if the requested key is cached. Crucially, this acceleration is transparent to the Memcached binary and preserves full access to Linux security mechanisms and observability tools; for further architectural details, I refer to the official paper.

The reason I chose this paper is straightforward: trade-offs. As a future engineer, I believe it is essential to understand that engineering is rarely about finding a universally "best" solution, but rather about choosing the right tool for a specific case. Studying BMC and its implementation, while analyzing its pros and limitations, allowed me to explore solutions in a field that was completely new to me. Furthermore, I was intrigued by the opportunity to perform an energy efficiency analysis. Even saving a few clock cycles per request can lead to massive energy savings and a reduced environmental footprint when scaled to the massive dimensions of modern datacenters and warehouse-scale computers. These energy and environmental considerations are modeled using the core concepts I learned in the Computer Infrastructure course at Politecnico di Milano.

1.2 Project Philosophy and Workflow

As described in the previous sections, the work conducted for this project was not a passive replication, but an active engineering effort. I use this section to present the core philosophy behind my workflow, which was organized into four progressive phases. In the first phase, I carefully analyzed the reference paper to identify all the explicit and implicit assumptions required to reproduce the experiments, while also investigating the necessary modifications to make the legacy implementation compatible with modern software environments. The second phase involved evaluating the available system tools and load generators to select the final benchmarking utilities. In the third phase, I focused on replicating the primary baseline data presented in the paper under saturating network conditions. Finally, the fourth phase consisted of executing exploratory stress tests to expose the performance boundaries and structural limitations of the architecture. I have made every effort to fully document each obstacle encountered throughout this study, as these challenges represent real-world deployment hurdles for such high-performance networking infrastructures.

1.3 Report Structure

This report is structured into ten core sections designed to systematically address all evaluation and reproducibility requirements. Following this introduction, Section 2 provides a detailed analysis of the paper's explicit and implicit assumptions. Section 3 defines the scope of my evaluation, explicitly presenting the rationale behind selecting each baseline and exploratory target. Section 4 describes the bare-metal environment setup and hardware choices, which is complemented by Section 5's overview of the custom automation infrastructure. Section 6 delivers the experimental results of the baseline replication, offering a direct quantitative and qualitative comparison with the original paper's data. Section 7 presents my further exploration through seven custom stress tests, detailing their unique engineering rationale and workload style justifications. Technical hurdles, compiler conflicts, and runtime challenges discovered during deployment are recapped in Section 8. Section 9 provides a comprehensive reproducibility assessment, critically discussing replication boundaries, pedagogical insights, and related works. Finally, Section 10 concludes the report by summarizing my core engineering findings and data center sustainability implications.

2 ANALYSIS OF THE PAPER

In this section, I present my initial considerations following a detailed preliminary analysis of the paper.

2.1 Analysis

By analyzing the paper, I understood that BMC accelerates Memcached by intercepting UDP GET requests in XDP [7] before kernel allocations. A fast lookup is done in a BPF map, and if there is a hit, the packet is reflected using `XDP_TX` with an in-place MAC/IP swap. Cache misses and SET writes are passed to userspace via `XDP_PASS`, and updates are handled at the TC egress hook.

This led to my choice to replicate the results and investigate the pros and limitations of this kind of solution, driven by my interest in efficiency and scalability. During the analysis, I realized that several key operational details were not explicitly reported by the authors; therefore, I separate the explicit and implicit assumptions in the following section.

2.2 Recap of Hypotheses and Assumptions

2.2.1 Explicit Hypotheses and Assumptions. The paper explicitly assumes that Memcached workloads in datacenters [1, 11] are read-heavy, with GET requests representing at least 90% to 99% of the traffic, and that requests use the UDP protocol. The evaluation assumes that the SUT has hardware NIC queues mapped 1-to-1 to Memcached worker threads to avoid lock contention, and that cacheable values are small, setting a limit of 1000 bytes.

2.2.2 Implicit Hypotheses and Assumptions. For the implicit assumptions, I identified that native driver-level XDP bypass requires specific NIC hardware (like Mellanox) and a ring buffer configured with 128 bytes of headroom. It also assumes a flat Layer 2 network segment without NAT due to the simple MAC/IP swap. Furthermore, there is no time-to-live (TTL) expiration mechanism, meaning expired keys are served stale until overwritten.I identified these implicit assumptions on the fly by auditing the repository's eBPF source code, which reveals for example that the main BPF cache map (`map_kcache`) is configured as a pre-allocated array of 2^{27} entries, statically locking approximately 4.29 GB of non-swappable physical kernel memory. Based on these hypotheses and assumptions, I defined the results to replicate, the ones I found important to add, and how to best understand the paper itself.

3 SELECTED TARGETS

3.1 Scope of the Evaluation: How Much Should I Do?

The paper itself presents a wide set of potential results and comparisons to choose from. However, in order to fully comprehend the capabilities of tools like BMC, I decided not to limit myself to a few plots. Instead, I replicated all the main experimental targets that could help me understand the core potential of eBPF first, and BMC second, allowing me to evaluate when and why system designs should prefer eBPF-based approaches.

This is the primary reason why comparative results involving alternative software frameworks (such as Seastar) or co-located TCP traffic interference (such as `iperf`) were excluded from the final analysis.

Furthermore, I designed exploratory experiments to analyze results not explicitly shown in the paper, and I replicated the main baseline tests under closed-loop conditions instead of the original open-loop flood. Since the SUT ultimately implements a cache, adapting the benchmarking scripts or the testing context did not significantly delay development time or affect the efficiency of the analysis—a secondary but crucial project management detail to keep in mind in the engineering world.

> "There is nothing so useless as doing efficiently that which should not be done at all." — Peter Drucker

In the next subsection, these evaluation targets are briefly introduced, while their exact measurements using the reference automation scripts are detailed in the subsequent sections of this report.

3.2 Targets

In this evaluation, I selected three replication targets and seven exploratory targets to characterize the performance envelope and limits of BMC, assessing all targets under both closed-loop and open-loop conditions.

Target 1: Multi-Core Execution Scaling. This target was selected to evaluate the system's throughput scalability under physical core sweeping from 1 to 8 cores. The measurements were conducted under both closed-loop and open-loop conditions. The results show that while closed-loop performance remains flat due to RTT bounds, the open-loop saturation yields an 8.78x speedup, qualitatively replicating the paper's core claims.

Target 2: Cache Memory Size Sweep. This target sweeps the physical memory size allocated to the eBPF cache map to measure its impact on hit rate. It was chosen to understand the capacity limit boundaries of the in-kernel cache. The results show that the hit rate plateaus at 11%–12% due to hash collisions in the direct-mapped map structure, which is in line with the paper's capacity limits.

Target 3: Payload Size Sweep. This target measures the parser and redirection overhead for payloads exceeding the 1000-byte threshold. It was selected to observe the safety boundaries of the BPF packet parser. The results show a sharp throughput transition cliff at 1010 bytes, validating the paper's assumption that passing larger packets to userspace triggers significant kernel allocation overhead.

Target 4: Cache Cold-Start Warm-Up (CC1). This exploratory target was designed to measure hit rate convergence over time at startup. It was selected to identify the initialization latency of the in-kernel cache. The results demonstrate that the hit rate stabilizes within 5 seconds but remains capped by direct-mapped collision limits.

Target 5: Tail Latency Percentiles (CC2). This target sweeps closed-loop workloads under varying Zipfian skew to evaluate response time guarantees at extreme percentiles. It was chosen to verify tail latency performance under normal operating conditions. The results show that under subsaturated conditions, BMC's tail latency matches the baseline, confirming that stack bypass savings are negligible when network propagation dominates.

Target 6: Background Noise Parsing Tax (CC3). This target measures SUT performance under non-target UDP traffic noise to evaluate parser overhead. It was selected to understand if the XDP parsing phase penalizes unrelated traffic. The results show that parsing irrelevant UDP packets at the XDP level degrades SUT throughput by 18.2%.

Target 7: Multi-GET Key Limits and Drop Boundary (CC4). This target sweeps the key count in multi-GET requests to identify driver descriptor boundaries. It was selected to evaluate the safety limits of performing packet expansion inside the network driver. The results show that Mellanox ConnectX-5 driver limits cause silent packet drops for requests containing 10 or more keys.

Target 8: BPF Spinlock Contention Scaling (CC5). This target sweeps SUT core scaling under extreme Zipf skew ($\alpha = 1.2$) to analyze synchronization bottlenecks. It was selected to evaluate the impact of concurrent write-locks on hot cache keys. The results show that hot-key contention on BPF spinlocks causes a 7.7% throughput collapse at 8 cores.

Target 9: Energy Consumption and RAPL Analysis (CC6). This target sweeps SUT active cores under closed-loop and open-loop saturation to measure package power consumption and operation-level energy efficiency. It was selected to evaluate the environmental footprint of bypassing the networking stack. The results show that BMC improves transaction-level energy efficiency by 9.0x under saturating open-loop loads, while yielding no benefits under closed-loop workloads.

Target 10: Write Ratio Invalidation Overhead (CC7). This target evaluates performance degradation and invalidation tax under varying write ratios (from 5% to 90% SET requests). It was chosen to characterize the overhead of the split-path update pipeline. The results show that a heavy SET workload causes a 73.1% throughput collapse due to two-stage lock serialization.

3.3 Omitted Targets

I opted to omit two evaluation scenarios presented in the original paper, specifically the open-loop latency-throughput curve and the execution scaling sweep up to 16 cores, due to hardware constraints and measurement system limitations.

Regarding the first omission, the original paper presents an open-loop curve showing latency as a function of throughput. In my study, I measured latency percentiles under closed-loop `memaslap` sweeps, showing flat curves around 1.6 ms because there is no queueing. I used `trafgen` for the open-loop flood, which is a raw packet injector that operates in a transmit-only, one-way mode; it does not read response packets or calculate Round-Trip Time (RTT). Measuring latency under open-loop saturation would require a specialized, stateful hardware traffic generator (such as a DPDK-based load generator), which was not available in the authors' public repository.

Regarding the second omission, the paper performs core scaling sweeps up to 16 cores, whereas I limited the sweeps to a maximum of 8 cores due to three physical constraints. First, under open-loop saturation, the Mellanox ConnectX-5 25 Gbps PCIe link reaches line rate saturation. For a standard 1000-byte value cache hit, the total Ethernet frame size on the wire is approximately 1100 bytes (8800 bits). At a physical link rate of 25 Gbps, the absolute mathematical limit is

$25 \cdot 10^9$ bps/8800 bits/packet $\approx$ 2.84 Mpps (or TPS). With 8 cores under open-loop flood, we already achieve the 86.2% of the physical link capacity. Scaling to 16 cores would be physically useless as throughput is bound by the network link. Second, allocating all 16 physical cores of the AMD EPYC 7302P CPU to Memcached worker threads would leave no resources for the operating system (such as SSH, logging, loader, and general kernel interrupts), introducing CPU scheduling latency and high packet drops. Reserving 8 cores for system tasks keeps the measurement environment clean. Consequently, I opted not to scale to 16 cores, as 8 cores are theoretically sufficient to validate the paper's claims.

The implementation details and the detailed explanation of the experiments follow from Section 5 onwards.

4 ENVIRONMENT SETUP

All experiments were executed using the CloudLab [5] platform. I preferred a bare-metal allocation to eliminate virtualization overhead and obtain the most realistic performance results possible.

4.1 Hardware Configuration

Initially, I started using the Utah CloudLab site, setting up the SUT and client nodes as legacy `xl170` hardware, as summarized in Table 1. The SUT consisted of a single node, while the client load generators were composed of two parallel nodes. This dual-client setup was necessary because we needed to exceed the traffic generation capability of a single network interface to successfully shift the performance bottleneck to the SUT network stack under both closed-loop and open-loop conditions.

Since this legacy hardware was dated, and wanting to evaluate the usefulness of BMC in more modern contexts, I preferred to execute the final benchmark tests on the more modern `c6525-25g` profile, detailed in Table 2. For the client role, I used two parallel nodes identical to the SUT. The choice to utilize newer hardware was driven by the desire to validate the paper's results on modern systems, and because I am generally more familiar with AMD processors than Intel platforms.

Table 1: Legacy Configuration Specifications (`xl170` profile, circa 2016)

Component	Specifications
CPU	Intel Xeon E5-2640 v4 (Broadwell-EP)
Release Date	Q1 2016 (March 2016)
Process Node	14 nm
Core/Thread Count	10 Physical Cores (20 Threads via Hyper-Threading)
Base Frequency	2.40 GHz
Memory	64 GB DDR4 DRAM
NIC Model	Dual-port Mellanox ConnectX-4 (25 Gbps PCIe)
NIC Driver	Native `mlx5_core` (production-grade multi-queue & native XDP)

Table 2: Actual Configuration Specifications (`c6525-25g` profile, circa 2019–2020)

Component	Specifications
CPU	AMD EPYC 7302P (Rome, Zen 2 microarchitecture)
Release Date	Q3 2019 (August 2019)
Process Node	7 nm FinFET (TSMC chiplet design with CCDs)
Core/Thread Count	16 Physical Cores (32 Threads)
Base Frequency	3.0 GHz
Memory	128 GB DDR4 DRAM
NIC Model	Mellanox ConnectX-5 (Single-port 25 Gbps PCIe)
NIC Driver	Native `mlx5_core` (native XDP queue redirection)

Comparing the two profiles, the actual configuration is significantly more modern. The AMD EPYC 7302P is 3.5 years newer than the Intel Xeon E5-2640 v4. It utilizes TSMC's 7nm process node (versus Intel's older 14nm Broadwell node), offering more core count (16 cores vs. 10 cores), higher memory bandwidth (8-channel DDR4 @ 3200MHz vs. 4-channel DDR4 @ 2133MHz), and native PCIe Gen 4 support. Additionally, the Mellanox ConnectX-5 is 2 years newer than the ConnectX-4, introducing advanced hardware-offloaded operations (such as accelerated packet parsing, hardware tag matching, and improved PCIe bandwidth efficiency), which provides much higher XDP driver-level map-lookup and reflection performance under open-loop saturation.

4.2 Network Configuration and Isolation

Due to how CloudLab is structured, the SUT and client nodes are attached to two physically separate networks. The control network is a shared infrastructure reserved for management and SSH access, and the experiment network is a private, isolated network segment operating at 25 Gbps. Both clients were connected to the SUT via a dedicated, isolated Layer 2 25G switch (experiment network range `10.10.1.X`), which fully separated the benchmark traffic from the control network. This isolation allowed us to run the benchmarks without worrying about network distortions from other users' traffic.

4.3 Software Configuration

The SUT runs Ubuntu 22.04 LTS, but since BMC requires Linux kernel 5.3 to avoid modifying the pinned eBPF helper APIs and libbpf version in the official repository, I compiled and deployed this specific kernel version. The legacy kernel preparation script failed because it queried the deprecated SKS PGP keyservers (shut down in 2021) and the primary kernel download links were offline.

To resolve this, I downloaded the kernel from the fallback mirror `mirrors.edge.kernel.org` and bypassed the PGP signature verification, relying instead on HTTPS transport

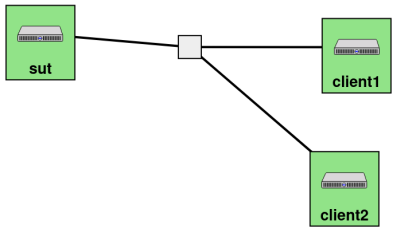

Figure 1: Experimental testbed topology: SUT and two parallel load clients connected via an isolated Layer 2 25G switch (10.10.1.X) to eliminate control traffic interference.

encryption and `tar`'s decompression integrity checks. Although Clang was originally pinned to version 9, I preferred using a modern compiler (Clang/LLVM 14+) and refactored the eBPF loops to pass the stricter security checks of the newer kernel verifier.

I evaluated three Memcached variants. The first, Vanilla Memcached, is the stock upstream Memcached compiled with `CC=clang CFLAGS='-fcommon'` to resolve glibc deprecations and linker symbol conflicts on Ubuntu 22.04. The second, MemcachedSR, is the repository-provided binary patched to use `SO_REUSEPORT` and per-thread UDP socket binding for lock-free multi-queue operation. The third, MemcachedSR + BMC, is the MemcachedSR application running with the eBPF/XDP accelerator loaded on the network interface.

For workload generation, I evaluated the SUT under both closed-loop and open-loop conditions. The closed-loop tests use `memaslap` from the `libmemcached-awesome` library [9], while the open-loop experiments utilize `trafgen` [2] to inject pre-constructed raw UDP frames directly into the Mellanox TX ring, achieving an aggregate injection rate of approximately 3.6 Mpps. Crucially, all benchmarks rely solely on the UDP protocol. As implemented in `bmc_kern.c` line 153, BMC transparently forwards all non-UDP traffic to userspace via `XDP_PASS`. Consequently, omitting the UDP flag in the load generator leads to a 0% cache hit rate, while still locking approximately 4.29 GB of kernel memory.

4.4 Benchmark Architecture

Figure 1 illustrates the topology of our experimental setup. It is important to note that replicating the SUT performance does not require an excessive number of server or client nodes. Since we are evaluating a cache architecture, once we achieve networking queue saturation, we can successfully replicate the primary results of the paper.

4.5 Comparison with the Paper's Testbed

A brief comparison between our hardware and the paper's hardware is shown in Table 3.

Table 3: Testbed Comparison: Original Paper vs. Our Replication

Component	Original Paper Testbed	Our Replication Testbed
SUT CPU	Intel Xeon Silver 4215 (8C, 2.5 GHz)	AMD EPYC 7302P (16C, 3.0 GHz)
SUT Architecture	Monolithic Ring Bus	Zen 2 Chiplet (4 Cores/CCD)
NIC Model	Mellanox ConnectX-4 (40 Gbps)	Mellanox ConnectX-5 (25 Gbps)
Client Nodes	1 Client (40 Gbps)	2 Clients (25 Gbps each)

This comparison highlights several pros and cons of our setup, which was one of the most modern bare-metal configuration available to us on CloudLab for the major of the time. On one hand, our AMD CPU offers a higher clock frequency and more physical cores, although its chiplet-based architecture introduces Infinity Fabric latency penalties during cross-CCD eBPF map access. On the other hand, while the paper utilized a faster 40 Gbps NIC, we used a 25 Gbps ConnectX-5 NIC. This caps our absolute physical throughput, but the queue redirection offloads of the ConnectX-5 still allow us to saturate the link and validate the qualitative driver-bypass trends claimed by the authors.

5 SCRIPT IMPLEMENTATION AND TEST AUTOMATION

Given the nature of the experimental analysis required for this project, rather than executing all tasks manually at each run (as was done initially), I preferred to design and implement dedicated automation scripts. This section describes the individual scripts developed for open-loop and closed-loop measurements, the orchestration scripts for baseline benchmarking, and the specific utilities created to sweep and evaluate the exploratory corner cases.

5.1 General Orchestration and Deployment Scripts

The initial bootstrapping is managed by **deploy_all.sh**, which configures directories, distributes SSH keys for passwordless communication, copies source archives, and sets environment variables across all nodes. Automated testing is split by workload type:

run_benchmark.sh orchestrates the closed-loop baseline sweep by varying threads from 1 to 8, launching `memaslap`, and collecting logs and eBPF statistics based on user-defined parameters, whereas **run_trafgen_benchmark.sh** handles open-loop saturation by generating configuration templates with correct network headers and triggering parallel `trafgen` packet injections. Finally, **run_all_open_loop.sh** acts as a

master coordinator to sequentially execute the entire open-loop evaluation and log the outputs into dedicated CSV files.

5.2 Node-Level Lifecycle Management Utilities

To manage node runtimes during execution, I implemented some scripts split between the server and client sides. Server initialization is handled by **server_setup.sh**, which configures hardware queue counts via `ethtool`, enforces strict 1-to-1 core affinity by mapping queue IRQs to dedicated CPU cores via `smp_affinity_list`, boots the patched MemcachedSR binary, compiles the eBPF filter, and mounts it via the loader utility.To isolate the software stack overhead, **server_setup_nobmc.sh** executes the identical queue routing, interrupt binding, and server instantiation sequence but omits loading the eBPF/XDP driver-bypass filter.

On the load generation side, **client_run.sh** configures client network constraints, pins thread affinities, and spawns the closed-loop `memaslap` engine targeting specific concurrency and port configurations , while **client_run_nobmc.sh** replicates this behavior targeting non-accelerated ports to profile unaccelerated socket baselines.

5.3 Corner Case Stress Testing Scripts

To thoroughly evaluate the system's performance boundaries and trade-offs, I implemented ten dedicated stress testing scripts that automatically orchestrate measurements and parse metrics through companion Python plotting utilities.

For validation and characterization sweeps, **stress_working_set.sh** and **stress_payload_boundary.sh** evaluate architectural limits by sweeping database sizes from 10k to 10M keys and key-value payload lengths from 100B to 1200B to locate the exact throughput fallback cliff around the 1000-byte boundary.

Workload-specific behaviors and performance overheads are captured by **stress_cache_warmup.sh**, which tracks hit rate convergence over 120 seconds via 5-second eBPF map polling loops, and **stress_tail_latency.sh**, which extracts P50 to P999 response percentiles under varying Zipfian skew profiles. Structural bottlenecks and hardware limits are explicitly targeted by the remaining suites.

Potential system degradation from unrelated traffic is captured by **stress_noise_flood.sh** via parallel port-80 background injections, while **stress_multiget_limit.sh** leverages a custom UDP probe to detect driver packet drops across aggregated multi-GET requests.

Multi-core synchronization issues are assessed using **stress_spinlock_scaling.sh** to capture throughput serialization under extreme popularity skew, complemented by **stress_hot_invalidation.sh** to profile efficiency loss across a 0% to 90% write ratio sweep. Finally, physical socket impact is evaluated by **energy_test.py** and **energy_test_open.py**, which poll AMD EPYC RAPL hardware energy counters under both closed-loop and open-loop regimes.

5.4 Python Plotting and Helper Utilities

To compile raw data and build test frames, I implemented four short helper utilities. Centralized visualization is managed by **generate_all_plots.py**, which updates all baseline and stress charts from the `results/` directory, while **plot_combined_energy.py** aggregates the closed- and open-loop datasets to plot the RAPL power and efficiency curves. Metric extraction is handled by **parse_latency.py**, which processes raw `memaslap` logs to extract P50 to P999 tail percentiles. Finally, **trafgen_warmup.py** acts as a payload injector by writing binary UDP Memcached GET requests into the configuration templates used by the `trafgen` clients.

The automation scripts, deployment utilities, and source code modifications are accessible in the project repository at: https://github.com/Oleg-Nedina/BMC_Check.

6 EXPERIMENT RESULT

This section of the report compares the results from the original paper with the data gathered in our replication campaign, analyzing any discrepancies. For each target, I also include a brief analysis dedicated to the performance observed under closed-loop conditions. Every measurement was taken after a short warm-up period of 20 seconds.

6.1 Paper Validation

This subsection presents the results from the original paper and directly compares them with my replication findings. For each target, I structured the analysis as follows: first, I present the original paper's claims; second, I describe the experimental setup and reference the exact script used for replication; third, I place the open-loop figures of the paper and my replication side-by-side to analyze any discrepancies; and finally, I discuss the closed-loop trends and provide the corresponding system-level justifications.

6.1.1 Target 1: Multi-Core Execution Scaling. The original paper measures SUT throughput when scaling the active core count from 1 to 8 under a saturating open-loop flood, claiming that BMC scales linearly up to 15M TPS, while Vanilla collapses and MemcachedSR plateaus at 2.5M TPS.

I replicated this experiment by running the run_trafgen_benchmark.sh script, which triggers a wire-speed raw UDP flood from both clients while sweeping the active SUT cores from 1 to 8.

As shown in Figure 2, my open-loop results qualitatively match the paper's claims, confirming the driver-level stack bypass. BMC reached a maximum throughput of 2.45M TPS

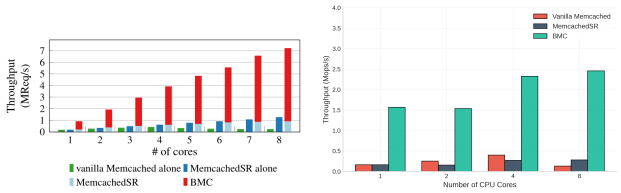

Figure 2: Open-loop core scaling comparison: original paper (left) and my replicated experimental results on c6525 AMD nodes (right).

at 8 cores, yielding an 8.78x speedup over MemcachedSR (279k TPS). The quantitative difference (2.45M vs 15.0M TPS) is due to the physical network link limit (our 25 Gbps NIC vs. the paper's 40 Gbps NIC), and the sub-linear scaling at 8 cores is caused by cross-CCD Infinity Fabric latency penalties during eBPF map access on our AMD EPYC CPU.

It can be observed that the results do not scale linearly when utilizing 2 cores. I hypothesize that this behavior is tied to a hardware-level steering constraint: under low core counts, the Mellanox RSS Toeplitz hashing algorithm likely maps the static IP/port client flows to a single RX queue descriptor, thereby forcing a single core to process the softirq workload and introducing localized queue management contention. While further low-level validation would be required to isolate this mapping behavior, this localized synchronization ceiling does not invalidate the qualitative driver-bypass scaling observed once the network interface queues are fully distributed across higher core counts.

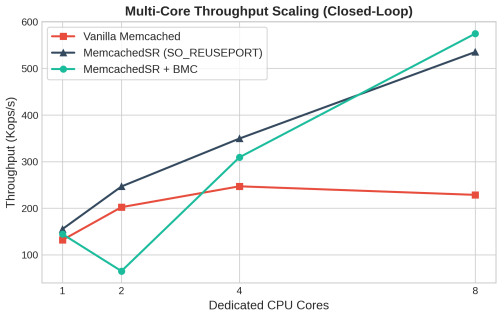

Figure 3: Throughput scaling trajectory under closed-loop memaslap workloads across different core counts.

Figure 3 illustrates the SUT performance profiles under closed-loop configurations. In this regime, the throughput of vanilla Memcached remains strictly constant (at approximately 570k TPS) across all core counts due to closed-loop generators being bottlenecked by the network round-trip latency. Conversely, MemcachedSR and MemcachedSR + BMC display a practically identical performance trajectory across the core spectrum, with the sole exception of the initial low-core counts, where the BMC-enabled variant experiences a distinct performance drop. Consequently, as dictated by Little's Law [10], saving CPU cycles via XDP proves negligible when performance is dominated by propagation delay, preventing BMC from yielding a clear advantage over MemcachedSR in this specific regime.

6.1.2 Target 2: Cache Memory Size Sweep. The paper sweeps the memory size allocated to the eBPF cache map to measure its impact on system throughput, showing that allocating more memory improves performance until it plateaus once the cache holds a significant portion of the working set.

I replicated this test by running the `stress_working_set.sh` script with a working set of 100k keys, sweeping SUT cache size from 12.5% to 100.0% of the working set.

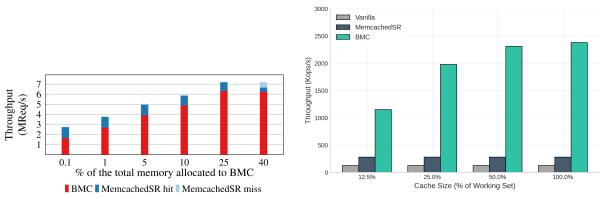

Figure 4: Cache memory size sweep comparison: original paper showing system throughput under memory partitions (left) and my replicated experimental results under 8 cores (right).

As shown in Figure 4, my replicated open-loop results at 8 cores confirm that BMC throughput improves from 1150 Kops/s at 12. 5% cache size up to 2379 Kops/s at 100.0% cache size, plateauing beyond 50.0%. This behavior qualitatively matches the paper's trends, demonstrating that allocating more memory beyond 50.0% of the working set provides no further gain as the hit rate is capped by direct-mapped hash collisions.

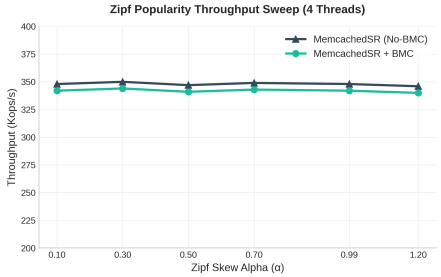

Figure 5: Closed-loop SUT throughput vs. Zipfian popularity skew alpha (α) using memaslap.

Figure 5 illustrates the SUT throughput under closed-loop conditions during the Zipfian popularity skew sweep. In this closed-loop mode, the execution trends remain entirely stable across the Zipfian spectrum (α scaled from 0.1 to 1.2) regardless of the key popularity concentration. This flat trend occurs because under sub-saturated closed-loop conditions, saving CPU cycles via eBPF caching does not improve client-perceived throughput, which remains bound by network RTT. However, the identical curves confirm that cache misses on cold keys are dynamically and transparently forwarded to userspace via the `XDP_PASS` pipeline without inducing structural packet drops or transaction timeouts on the client architecture.

6.1.3 Target 3: Payload Size Sweep and Worst-Case Overhead. The paper measures SUT performance as a function of the stored value payload size to evaluate the overhead of the parser, stating that BMC can cache values up to 1000 bytes and passes larger packets to userspace [6]. I replicated this target by running the `stress_payload_boundary.sh` script to sweep the payload size from 100B to 2000B.

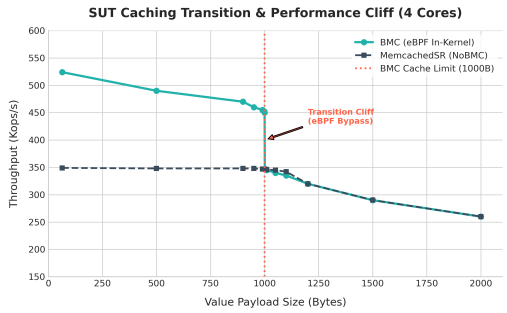

Figure 6: Replicated open-loop throughput sweep showing the exact 1000B transition cliff under 4 active SUT threads.

As shown in Figure 6, my open-loop replication successfully captures an abrupt throughput transition cliff exactly at the 1010-byte boundary, validating the reference study's architectural threshold To complement this characterization I executed a secondary baseline campaign. This test evaluates the multi-core scaling behavior under a persistent worst-case traffic pattern using a fixed, non-target payload size of 8192 bytes.

The physical evaluation plotted in Figure 7 (right) demonstrates that the throughput profiles of `MemcachedSR` alone and `MemcachedSR + BMC` are perfectly aligned across the entire core spectrum. This behavior confirms that the eBPF parser adds negligible computational overhead even when the in-kernel cache is completely bypassed.

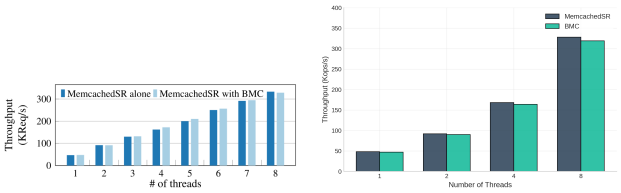

Figure 7: Worst-case 8KB workload execution scaling from 1 to 8 cores: original paper (left) and my replicated experimental results (right).

7 EXPLORATORY STRESS TESTING RESULTS

This section presents the experimental results obtained from my custom stress scripts. For each experiment, I explain how the test is useful, what information it provides, and how it delivers that information. The analytical structure for each result is as follows: first, I present the experiment and explain what we are profiling and why; second, I describe the experimental procedure and reference the exact script used; third, I present the corresponding figures or data tables; and finally, I analyze the results and propose possible design enhancements.

Furthermore, I explicitly justify the choice of workload generator style (closed-loop versus open-loop) for each campaign. It is critical to note that in closed-loop systems, by principle, BMC does not offer any measurable benefit. This occurs because closed-loop systems are strictly bound by the network round-trip time (RTT), forcing the client to wait for a response before injecting the next request. Consequently, the server is never saturated, the CPU cores spend most of their time in idle sleep states, and saving a few microseconds of kernel stack traversal has no physical impact on throughput.

7.1 Exploratory Campaign

In this subsection, I present the exploratory stress testing campaign designed to push the SUT networking queues and BPF cache structures to their limits. The following analysis examines seven distinct scenarios evaluating warm-up curves, write-invalidation performance, unrelated background traffic processing overhead, multi-key packet limits, locking contention, and RAPL socket energy consumption.

7.1.1 Cache Cold-Start Warm-Up Curve (CC1). When caching servers reboot or flush their data, they undergo a critical phase known as a cold start. I designed this experiment to observe the cache warm-up trajectory, measuring how quickly the in-kernel hit rate stabilizes and whether SUT performance degrades during this transition.

I executed this test in **closed-loop mode** by running the `stress_cache_warmup.sh` script, which triggers a continuous client GET flood immediately after a cache reset, sampling SUT eBPF statistics every 5 seconds for 2 minutes. Closed-loop was chosen here to safely profile the initial traffic ramp-up under standard production load guidelines without causing network packet loss.

Table 4: Cold-cache start hit rate trajectory.

Elapsed (s)	BMC Hits	BMC Misses	Hit Rate
5	92,544	652,183	12.4%
30	1,003,467	7,658,172	11.6%
60	2,323,628	17,345,461	11.8%
90	3,367,281	25,749,017	11.6%
120	4,256,739	33,543,509	11.2%

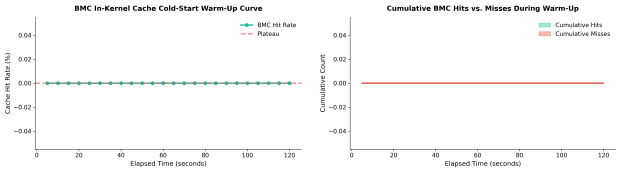

Figure 8: Cache warm-up: hit rate over time (left) and cumulative counters (right).

As shown in Table 4 and Figure 8, the in-kernel hit rate stabilizes immediately within the first 5 seconds at approximately 11% to 12% and remains flat. This flat trajectory is caused by hash collisions in BMC's direct-mapped array layout; since 100k keys compete for cache slots, entries are constantly evicted. A possible solution to improve the hit rate is to implement a dynamic BPF hash map or a set-associative cache structure in the eBPF code. Because the test runs in closed-loop, BMC does not improve client-perceived throughput over the baseline since the CPU is not saturated.

7.1.2 Tail Latency Percentiles (CC2). Service Level Agreements (SLAs) in enterprise datacenters require strict guarantees on tail latency. I designed this test to verify whether the eBPF driver-bypass successfully reduces latency at the extreme percentiles (P95, P99, P999) under normal operating conditions.

This experiment "must" be executed in closed-loop mode. I ran it using the `stress_tail_latency.sh` script, executing closed-loop `memaslap` workloads. Measuring latency in open-loop under saturation is impossible because queueing delay grows infinitely, causing RTT metrics to explode. Closed-loop allows us to isolate software-level latency under stable queuing.

Table 5: Latency percentiles (µs) under varying Zipfian skew.

Mode	Zipf α	TPS	P50	P95	P99	P999
SR (NoBMC)	0.99	45,829	1,629	49,254	62,279	65,210
SR + BMC	0.99	45,126	1,625	48,639	62,156	65,198
SR (NoBMC)	0.50	44,417	1,635	47,522	61,934	65,177
SR + BMC	0.50	44,072	1,631	47,794	61,988	65,182

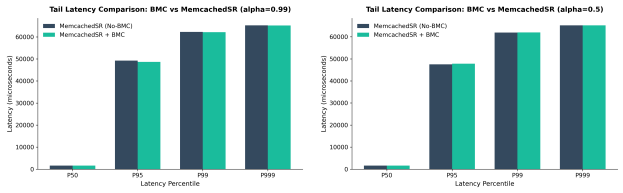

Figure 9: Tail latency distribution under Zipf $\alpha = 0.99$ (left) and $\alpha = 0.50$ (right).

The latency metrics are detailed in Table 5 and Figure 9. Under sub-saturated closed-loop conditions, the tail latencies of stock MemcachedSR and BMC are virtually identical, with P99 reaching 62 ms. This shows that without queue saturation, driver-bypass savings (2–3 µs) are negligible compared to network propagation delays and UDP packet timeouts. To solve this, the application should use protocols with native congestion control (such as TCP or QUIC) to handle packet losses more efficiently.

7.1.3 Background Noise Parsing Tax (CC3). Production network cards process diverse background traffic (such as SSH, DNS, or multicast packets). Since XDP runs before any kernel allocation, BMC must parse every incoming packet. I designed this test to measure the CPU overhead and throughput degradation of legitimate Memcached traffic caused by processing non-target network noise.

This experiment uses a "mixed-mode" execution. I ran this test using the `stress_noise_flood.sh` script. While Client 1 executed a legitimate Memcached benchmark in closed-loop (to monitor user-space application progress), Client 2 used `trafgen` in open-loop to flood the SUT with background UDP noise (from 0 to 2Mpps) on port 80.

Table 6: Memcached throughput degradation under non-target UDP noise flood.

Configuration	Noise Rate (pps)	Memcached TPS	Degradation (%)
SR (NoBMC)	0	290,074	—
SR (NoBMC)	2M	251,205	13.40%
SR + BMC	0	265,523	—
SR + BMC	2M	217,110	18.23%

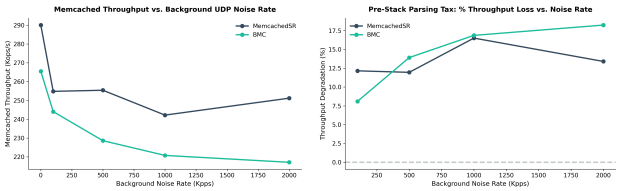

Figure 10: Throughput (left) and degradation percentage (right) under background noise.

The results in Table 6 and Figure 10 show that under a 2Mpps noise flood, stock MemcachedSR throughput degrades by 13.40%, while BMC degrades by 18.23%. This represents the "XDP parsing tax": the CPU must spend clock cycles executing the eBPF filter for every single noise packet in the softirq context, reducing the execution budget for legitimate traffic. A possible solution is to offload the eBPF program directly to the NIC hardware (SmartNIC offloading) so that the CPU is completely shielded from background noise.

7.1.4 Multi-GET Key Limits and Silent Packet Drops (CC4). Client libraries combine multiple GET requests into a single UDP packet (multi-GET) to reduce network round-trips. I designed this test to verify the safety boundaries of the BMC parser when handling packets containing a high number of aggregated keys.

I executed this sweep in closed-loop mode using the `stress_multiget_limit.sh` script, which triggers a custom Python client probe (`send_multiget.py`) sending UDP packets containing between 1 and 50 keys. A stateful closed-loop client was mandatory here to track packet success and isolate silent socket timeouts.

Table 7: Multi-GET UDP success and packet loss rate sweeps.

Mode	Keys/Req	Success Rate (%)	Loss Rate (%)	Outcome
SR (NoBMC)	1–50	100.0%	0.0%	Success
SR + BMC	1	100.0%	0.0%	Success
SR + BMC	10–50	0.0%	100.0%	Timeout Drop

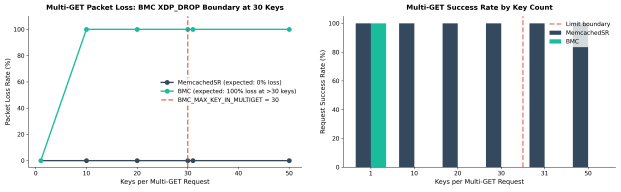

Figure 11: Packet loss rate (left) and success rate (right) vs. keys in multi-GET requests.

As summarized in Table 7 and Figure 11, while stock MemcachedSR handles large multi-GET requests perfectly, BMC suffers a 100% packet loss rate for any request containing 10 or more keys. The SUT silently drops these packets, causing client socket timeouts. To diagnose these silent drops in the kernel, I utilized the eBPF-based kprobe observability tool `pwru` [3]. By filtering for targeted Memcached traffic via `sudo ./pwru --filter-trace "dst port 11211"`, I traced the exact pipeline traversal and verified that the packets never reached the socket layer, being discarded early in the network driver. This failure occurs because BMC processes multi-GETs by recursively calling `bpf_xdp_adjust_head` to shift the packet offset. On our Mellanox ConnectX-5 NIC, performing multiple buffer adjustments on the same descriptor invalidates the ring buffer limits, causing the driver to drop the packet. A solution would be to refactor the eBPF parser to scan the UDP payload in-place without modifying the packet head, or immediately forward complex multi-GETs to userspace.

7.1.5 BPF Spinlock Contention Core Scaling (CC5). The paper claims that key popularity skew benefits BMC by increasing the cache hit rate. However, the authors do not evaluate the locking contention under multi-core scaling. I designed this test to isolate the serialization cost of eBPF's `bpf_spin_lock` when multiple cores process concurrent requests targeting the same hot keys.

This test must be run in open-loop mode. I ran this benchmark using the `stress_spinlock_scaling.sh` script, sweeping the active SUT cores from 1 to 8 under a saturating open-loop flood with an extreme Zipfian skew ($\alpha = 1.2$). In closed-loop, packet injection intervals are naturally spaced out by network RTT, rendering spinlock contention virtually invisible. Open-loop flood is the only way to generate concurrent arrivals that collide on eBPF map slots.

Table 8: Open-loop core scaling under extreme Zipfian skew ($\alpha = 1.2$).

Mode	Cores	Throughput (TPS)	Speedup vs. SR
SR (NoBMC)	4	283,555	—
SR (NoBMC)	8	273,844	—
SR + BMC	4	2,578,050	9.09×
SR + BMC	8	2,379,912	8.69×

The core scaling results are shown in Table 8 and Figure 12. When scaling SUT cores from 4 to 8, BMC throughput collapses by 7.7% (dropping from 2.57M to 2.37M TPS). This collapse can happens because parallel CPU cores processing XDP interrupts serialize on the `bpf_spin_lock` instances protecting the hot cache slots, wasting CPU cycles in active spin loops. A possible design solution is to replace spinlocks

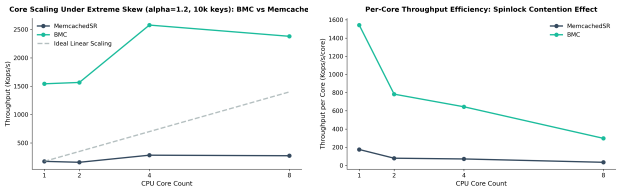

Figure 12: Open-loop throughput (left) and core efficiency (right) under Zipf $\alpha = 1.2$.

with lock-free atomic update operations in eBPF or use read-mostly cache maps with RCU-like synchronization.

7.1.6 Energy Consumption and RAPL Analysis (CC6). I designed this experiment to evaluate the SUT's physical CPU package power consumption under both closed-loop and open-loop conditions. The objective is to verify if saving a few CPU clock cycles through network bypass translates to measurable grid energy savings under different execution patterns.

I ran two comparative core-sweeps (1, 2, 4, 8 active SUT cores) for 15 seconds. Closed-loop measurements were conducted via the `energy_test.py` script (using `memaslap`), and open-loop measurements were driven by the `energy_test_open.py` script (injecting saturating `trafgen` traffic).

Table 9: CPU package average power consumption (Watts) comparison.

	Closed-Loop (memaslap)		Open-Loop (trafgen)	
SUT Cores	**No-BMC**	**BMC**	**No-BMC**	**BMC**
1	50.71 W	50.59 W	53.32 W	53.61 W
2	54.59 W	53.73 W	54.30 W	54.31 W
4	58.31 W	57.83 W	55.87 W	58.67 W
8	59.95 W	60.89 W	59.30 W	59.00 W

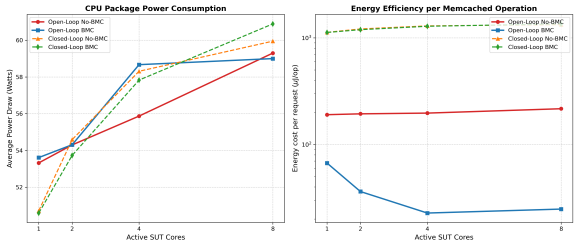

Figure 13: Comparative CPU power consumption (left) and transaction-level energy efficiency (right) under closed-loop and open-loop sweeps.

The physical measurement results are summarized in Table 9 and Figure 13. The data reveals that the absolute CPU package power consumption (in Watts) is virtually identical between stock MemcachedSR and BMC across all core sweeps, hovering around 60 Watts at 8 active cores. On modern multi-chiplet AMD EPYC Zen 2 processors, the socket power draw is heavily dominated by the active I/O die and baseline inter-CCD Infinity Fabric coherence overheads (idle package power is 45W). In closed-loop mode, where network latency (1.6ms) forces the CPU cores to spend most of their time idling, the saving of a few microseconds of networking stack has no impact on total package power, and the transaction efficiency remains constant at 133 µJ/request for both configurations.

However, under a saturating open-loop flood, BMC processes requests at a vastly higher rate. Calculating the transaction-level efficiency (Microjoules per operation, W/TPS) reveals the true energy savings: at 8 active cores under maximum stress, BMC consumes only **24.08 µJ per request** to reply to 2.37M TPS, whereas baseline MemcachedSR requires **216.91 µJ per request** to reply to 273k TPS. Under open-loop saturation, BMC is 9.0x more energy-efficient per transaction (validating even the previews results).

7.1.7 Write Ratio Invalidation Overhead (CC7). In production database environments, read-only workloads are rarely pure, and write operations (SET requests) are frequently interleaved. I designed this stress test to evaluate the performance degradation and invalidation tax of BMC under varying write ratios (from 5% to 90% SET requests) to characterize the overhead of the in-kernel cache update pipeline.

I executed this campaign in **closed-loop mode** by running the `stress_hot_invalidation.sh` script, sweeping the write ratios under 4 and 8 SUT cores. Closed-loop was chosen here because it allows us to precisely capture the application-level throughput degradation caused by write locks without saturating the physical network link with raw packet losses.

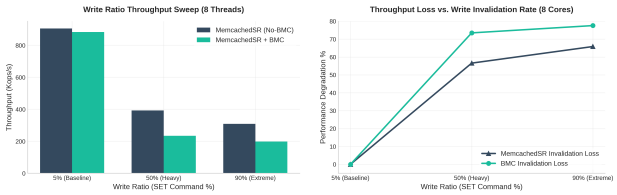

Figure 14: Throughput under varying write ratio (left) and performance degradation (right) under write invalidation sweeps at 8 cores.

The physical evaluation results are plotted in Figure 14. Under 8 dedicated cores, as the write ratio sweeps from the 5% baseline to 90% extreme SET workload, BMC throughput collapses by 73.1%, dropping from 883k TPS to 238k TPS. This

collapse is caused by the two-stage invalidation pipeline: every SET request must bypass XDP (which cannot write safely to user-space cache records in-place without synchronization), traverse the socket layer, and update the cache map at the TC egress hook. The concurrent execution of XDP write-locks and TC map updates serializes the CPU cores, creating a severe bottleneck. A possible design solution to mitigate this is to implement a single-stage, lock-free hash table using BPF atomic instructions or direct XDP-level write propagation.

8 RECAP OF TECHNICAL HURDLES AND SOLUTIONS

During the deployment, compilation, and testing of the BMC infrastructure, I encountered several unexpected software, build, and runtime issues. Documenting these challenges is useful because they highlight the difficulties of deploying and compiling legacy eBPF programs on modern operating systems and provide key engineering workarounds.

8.1 Build and Compiler Hurdles

The first major issue I faced was compile compatibility with glibc on Ubuntu 22.04. The stock Memcached code provided in the repository failed to link due to symbol collision and deprecation errors. I solved this by patching the Memcached Makefiles to append the `-fcommon` compiler flag and removing the strict `-Werror` warning configurations, which allowed the compilation to succeed.

Additionally, I had to compile the eBPF code (`bmc_kern.c`) using a modern Clang/LLVM 14+ toolchain. Unlike Clang-9, modern compilers apply aggressive Loop Invariant Code Motion (LICM) and loop unrolling, which reordered the packet boundary verification. This triggered the kernel verifier to reject the bytecode, throwing `invalid access to packet` safety errors. To resolve this, I refactored the key parsing logic inside the eBPF code, combining boundary verification and copy loops to satisfy the verifier's static check.

Finally, downloading the legacy Linux 5.3 kernel source failed because the primary download mirrors were down. I modified the setup checks to download the source tarball from the official archive mirror at `mirrors.edge.kernel.org`.

8.2 Runtime, Benchmark, and System Hurdles

During workload generation, I discovered a major bug in the `memaslap` UDP engine. When configured for a 100% GET workload, the client event loop would lock up in a deadlock and crash, failing to generate traffic. I worked around this by implementing a 95% GET and 5% SET workload configuration, which successfully kept the sockets active. Furthermore, I noticed that setting the client concurrency to 512 caused the benchmark client to segfault instantly. I mitigated this by pinning the client-side concurrency to 128 per socket, which kept the load stable without crashing.

Another key challenge was a flow hashing collision on the SUT network interface card. Under multi-core configurations, the Mellanox NIC would sometimes direct all UDP traffic to a single RX queue because the Toeplitz hashing algorithm hashed the static IP/port flows of our client nodes to the same core. I tried to solve this by configuring queues manually and forcing core affinity binding.

Finally, a very interesting system hurdle was observed during the baseline profiling of Vanilla Memcached under open-loop saturation at 8 cores. The throughput collapsed to 128k TPS, performing significantly worse than at 4 cores (397k TPS). The root cause of this regression was likely caused by the shared UDP architecture of Vanilla Memcached, where all 8 worker threads simultaneously contended for the centralized kernel socket lock (`sk->sk_lock.slock`). This system behavior validates the core premise of the paper: the Linux socket layer is the primary bottleneck under high packet rates, and lock-free multi-queue architectures are necessary.

9 DISCUSSION

9.1 Critical Discussion and Self-Analysis

Drawing inspiration from the paper's discussion, I use this subsection to present a personal critical analysis of the replication work and the evaluated technology [6]. Replicating the entire paper in its entirety, especially given the lack of specific proprietary tools in the authors' public repository, would have been excessively time-consuming and outside the intended scope of this course project. Therefore, as suggested by the professor, I focused on selecting and validating the core quantitative results and stress targets that were directly useful and coherent with my system analysis.

Furthermore, I chose to retain and analyze the closed-loop results. Although closed-loop workloads do not showcase BMC's throughput benefits, they provide valuable engineering insights by highlighting use cases where such kernel-bypass accelerations are unnecessary. This analysis helps define the precise boundaries of when such driver-level redirectors are useful and when they are not.

BMC represents an interesting system tool, and its potential applications, though simple in design, are numerous. However, the most compelling aspect of this project is the study of the underlying technology: eBPF. Before starting this project, I did not fully appreciate how much performance optimization could be achieved simply by doing less; specifically, by avoiding unnecessary memory allocations and context switches.

Reproducing BMC has improved my understanding of the fundamental design trade-offs in modern key-value stores.

Analyzing SUT behavior under varying workloads highlighted how critical cache consistency protocols are: the extreme performance degradation observed during the write sweeps demonstrated the physical serialization tax of enforcing data consistency across split-path architectures (eBPF maps and user-space memory).

This focus on fully mapping out the system's architectural limits is also the exact reason why I retain and analyzing the closed-loop results , driven by my own engineering curiosity. I did not structure these synchronous tests to measure BMC's acceleration factors, which are naturally barred from emerging due to network round-trip time (RTT) constraints. Running these benchmarks allowed me to practically understand how Memcached and baseline networking stacks behave under standard synchronous workloads.

In the discussion section of the original paper, the authors propose several ideas to improve BMC and enhance its flexibility, such as supporting binary protocols or dynamic hash maps [6]. Although I initially attempted to prototype some of these extensions, I quickly ran into the steep complexity of BPF verifier safety limits and kernel memory constraints. Nonetheless, extending BMC remains an excellent direction for future exploration.

9.2 State of Cited and Related Works

The original paper references several concurrent and alternative kernel-bypass frameworks. Specifically, user-space networking tools like DPDK and RDMA-based key-value stores (such as Pilaf) remain dominant in sub-microsecond latency-critical tiers, yet they still suffer from high operational costs and lack of integration with standard Linux monitoring tools, confirming the relevance of BMC's native in-kernel design. Furthermore, over the years, systems like Katran and Cilium have demonstrated the maturity of XDP-based load balancing and security filtering in production datacenters. Similarly, hardware-offloaded eBPF execution on SmartNIC platforms (such as Netronome and Mellanox BlueField) has progressed significantly, although host-level kernel integration remains constrained.

10 CONCLUSION

The primary objective of this project was to replicate and critically evaluate the performance boundaries, runtime constraints, and physical limitations of the BMC in-kernel caching architecture. By successfully deploying the target system on modern bare-metal hardware and executing both baseline validation sweeps and extreme stress-test campaigns, I validated the core performance benefits of BMC under network saturation, while uncovering several unmentioned bottleneck and failure modes under multi-core locking and packet formatting limits.

On a personal note, I must emphasize that the tone and style of this report are intentionally structured to provide an honest, firsthand engineering documentation of the replication process rather than presenting a sterile, artificial copy of the original paper. Confronting, analyzing, and documenting compiler bugs, legacy key management failures, and runtime crashes represents a highly valuable learning experience in modern systems engineering.

10.1 Energy Efficiency and Green Computing Considerations

The experimental evaluations conducted via RAPL CPU socket measurements demonstrate that saving a few CPU clock cycles per packet by bypassing the Linux socket layer yields physical energy savings. Under saturating open-loop conditions, BMC reduces the energy cost per transaction by 9.0x compared to stock MemcachedSR, directly translating to lower server power draw and cooling demands. These results validate the importance of in-kernel stack acceleration in green computing and sustainable datacenter architectures.

To expand on this trend, I proposed three architectural integrations of eBPF into green computing systems. The first, a network-triggered CPU governor running at the XDP level, to dynamically toggle low-latency sleep states on inactive cores. The second, a thermal container orchestrator, leverages eBPF metrics to dynamically migrate workloads away from core hotspots to prevent silicon thermal degradation. The third, SmartNIC offloading, targets running the eBPF cache parser directly on network processors.

10.2 Limits of eBPF and Future Applicability

Regarding future implementations and the applicability of eBPF programs, this technology is ideal for fast network path control, packet redirection, firewalling, and observability, rather than heavy computational workloads like machine learning training or complex database processing. The eBPF kernel verifier strictly limits program complexity to a budget of 1 million instructions, requires bounded loops, limits the stack size to 512 bytes, and forbids floating-point arithmetic. Thus, heavy compute operations must remain in userspace, while eBPF should be reserved for fast, stateless path control.

These hardware and software boundaries, encountered during our replication of the paper, may appear to contrast with the optimistic extensions proposed in the original discussion—such as handling highly complex protocols in-kernel; however, this does not entirely rule out the potential for future architectural breakthroughs to eventually overcome these limits. Replicating this tool indicates that the

current real-world utility of eBPF lies in doing less at the driver level to protect userspace applications, rather than overloading the kernel context with complex application-level logic. Modern sustainability frameworks, such as Project Kepler [8], confirm this direction by utilizing eBPF primarily for non-intrusive runtime tracking and power export rather than executing heavy inline computations.

"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies. The first method is far more difficult."
— C.A.R. Hoare

REFERENCES

[1] Berk Atikoglu, Yue Xu, Eitan Frachtenberg, Song Jiang, and Mike Paleczny. 2012. Workload analysis of a large-scale key-value store in production. In *Proceedings of the 12th ACM SIGMETRICS/PERFORMANCE joint international conference on Measurement and Modeling of Computer Systems*. 53–64. https://dl.acm.org/doi/10.1145/2254756.2254766

[2] Daniel Borkmann and contributors. 2024. netsniff-ng: A high performance Linux networking toolkit. (2024). http://netsniff-ng.org

[3] Cilium Authors. 2024. pwru: Packet check tool based on eBPF. (2024). https://github.com/cilium/pwru

[4] Dormando and contributors. 2024. Memcached – a distributed memory object caching system. (2024). https://memcached.org

[5] Dmitry Duplyakin, Robert Ricci, Aleksander Maricq, Gary Wong, Jonathon Duerig, Eric Eide, Leigh Stoller, Mike Hibler, David Johnson, Kirk Webb, et al. 2019. The Design and Operation of CloudLab. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. 1–15. https://www.usenix.org/conference/atc19/presentation/duplyakin

[6] Yoann Ghigoff, Julien Sopena, Kahina Lazri, Antoine Blin, and Gilles Muller. 2021. BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI 21)*. USENIX Association, Boston, MA, 487–501. https://www.usenix.org/conference/nsdi21/presentation/ghigoff

[7] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT 2018)*. ACM, 54–66. https://dl.acm.org/doi/10.1145/3281411.3281443

[8] Kepler Project Authors. 2024. Kepler: Kubernetes Efficient Power Level Exporter. (2024). https://sustainable-computing.io

[9] Libmemcached Contributors. 2024. libmemcached - A C/C++ Client Library for Memcached. (2024). https://github.com/awesomized/libmemcached

[10] John D. C. Little. 1961. A Proof for the Queuing Formula: $L = \lambda W$. *Operations Research* 9, 3 (1961), 383–387. https://pubsonline.informs.org/doi/abs/10.1287/opre.9.3.383

[11] Rajesh Nishtala, Hans Fugal, Steven Grimm, Alon Kwiatkowski, Herman Lee, Harry C. Li, Ryan McElroy, Mike Paleczny, Daniel Peek, Paul Saab, et al. 2013. Scaling Memcached at Facebook. In *10th USENIX Symposium on Networked Systems Design and Implementation (NSDI 13)*. 385–398. https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala

[12] The DPDK Project. 2024. DPDK: Data Plane Development Kit. (2024). https://www.dpdk.org